

校正 API フェーズ

校正 API フェーズ

LangChain + Gemini による日本語校正エンドポイントの実装

Hyonun Kin

2026-04-29

目的・背景

phase 2 までで「ログインしたユーザーを識別する」基盤が整った。本フェーズではアプリの中核機能である**校正 API**を実装する。

ユーザーがメール本文をテキストエリアに入れて送信すると、Gemini が以下の3観点で評価し、修正案と総合スコアを返す：

1. 誤字脱字 (typo)
2. 敬語の適切性 (取引先向けを想定) (keigo)
3. 自然な日本語表現 (expression)

設計判断として、**校正結果は永続化しない** (architecture.md の方針通り)。社内15人規模で「履歴を見たい」要件が無かったため、DB に corrections テーブルを持たない判断をした。これにより `/result/{id}` のような別ページも作らず、`/` の textarea の下に結果を直接表示する構成になる。

要件

機能要件：

- ・認証済みユーザーが `POST /api/correct` でテキストを送ると、校正結果が返る
- ・結果は `corrected` (修正後本文) / `score` (0-100) / `issues` (指摘リスト) の構造化 JSON
- ・認証必須 (Cookie の JWT が無効なら 401)
- ・校正結果は DB に保存しない

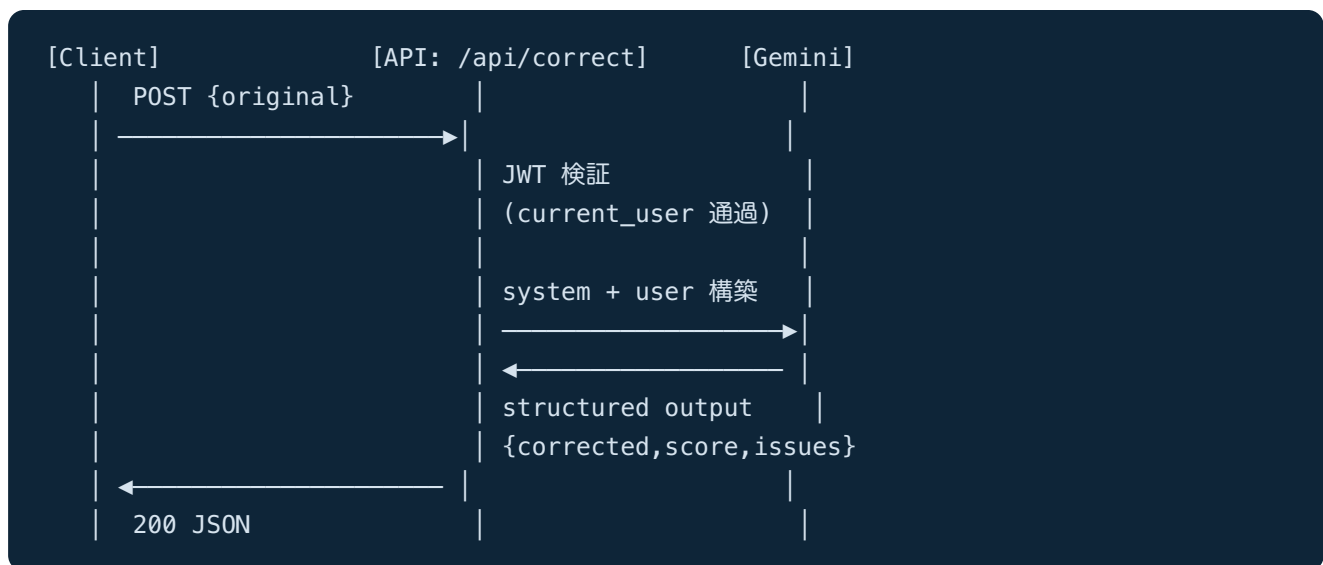
非機能要件：

- ・モデルは `gemini-2.5-flash-lite` (環境変数 `GEMINI_MODEL` で切替可)

- LangChain 経由で呼ぶ（プロバイダ切替時の影響を局所化）
- プロンプトは外部ファイル（`prompts/correct.yaml`）で管理し、本番運用時に再ビルド無しで調整可能にする
- テストは実 Gemini を呼ばない（API キー無し / ネットワーク不要 / 高速）

設計

シーケンス図



スキーマ

`schemas.py` に追加：

```

class IssueItem(BaseModel):
    category: str # "typo" | "keigo" | "expression"
    span: str # 原文の該当部分
    suggestion: str # 修正案
    reason: str # 修正理由

class CorrectIn(BaseModel):
    original: str

class CorrectOut(BaseModel):
    corrected: str
    score: int # 0-100
    issues: list[IssueItem]
  
```

LangChain × 構造化出力

`ChatGoogleGenerativeAI(...).with_structured_output(CorrectOut)` を使うと、

- 内部で Pydantic スキーマ → Gemini の関数呼び出し用 JSON Schema に変換
- 応答を自動で `CorrectOut` インスタンスにパース・バリデーション

→ 手書きの JSON parse / try-except / バリデーションが不要。

依存関数パターン（テストの差し替え点）

`api/ai.py` 末尾で `get_corrector()` を返す薄い依存関数を定義：

```
def get_corrector() -> Corrector:
    return correct_text
```

エンドポイント側で：

```
async def correct(
    body: CorrectIn,
    user: User = Depends(current_user),
    corrector: Corrector = Depends(get_corrector),
) -> CorrectOut:
    return await corrector(body.original)
```

テスト側で `app.dependency_overrides[get_corrector] = lambda: _fake_corrector` と差し替えるだけで、Gemini を一切呼ばずに endpoint の挙動を検証できる。

遅延初期化（lazy chain）

`_chain` を `None` で宣言し、初回 `correct_text` 呼び出し時に `_build_chain()` で組み立てる：

```
_chain: Any = None

async def correct_text(original: str) -> CorrectOut:
    global _chain
    if _chain is None:
        _chain = _build_chain()
    ...
```

理由：テスト環境で `GOOGLE_API_KEY` が未設定でも `import api.ai` が成功するようにするため。テストでは `correct_text` 自体が呼ばれないので `_build_chain()` も実行されない。

実装

ファイル構成

phase 3 で追加・変更したファイル：

```
backend/
├── api/
│   ├── ai.py           # 新規：Gemini 呼び出し + get_corrector 依存
│   ├── schemas.py      # IssueItem / CorrectIn / CorrectOut 追加
│   ├── config.py       # google_api_key / gemini_model 追加
│   └── main.py         # POST /api/correct を追加
├── prompts/
│   └── correct.yaml     # system / user の中身を実装
├── tests/
│   └── test_correct.py  # 新規：mock テスト 3 件
└── .env.example        # GOOGLE_API_KEY を追加
```

主要コード (ai.py)

```
_PROMPT_PATH = Path(__file__).parent.parent / "prompts" / "correct.yaml"
_PROMPTS = yaml.safe_load(_PROMPT_PATH.read_text())

_chain: Any = None

def _build_chain() -> Any:
    llm = ChatGoogleGenerativeAI(
        model=settings.gemini_model,
        google_api_key=settings.google_api_key or None,
    )
    return llm.with_structured_output(CorrectOut)

async def correct_text(original: str) -> CorrectOut:
    global _chain
    if _chain is None:
        _chain = _build_chain()
    messages = [
        {"role": "system", "content": _PROMPTS["system"]},
        {"role": "user", "content": _PROMPTS["user"].format(original=original)},
    ]
    return await _chain.ainvoke(messages)

def get_corrector() -> Corrector:
    return correct_text
```

プロンプト (prompts/correct.yaml)

```
system: |
    あなたは日本語ビジネスメール校正アシスタントです。
    入力テキストを以下の観点で評価し、JSON で結果を返してください：
    1. 誤字脱字 (category: "typo")
    2. 敬語の適切性 (取引先向けを想定) (category: "keigo")
    3. 自然な日本語表現 (category: "expression")

    出力フィールド：
    - corrected: 指摘事項を反映した修正後テキスト
    - score: 0-100 の総合スコア (指摘0件なら 100)
    - issues: 指摘リスト (category / span / suggestion / reason)

user: |
    {original}
```

検証

項目	結果
<code>uv run pytest -v</code>	14 件 pass (auth 7 + initial_admin 3 + smoke 1 + correct 3)
<code>test_correct_requires_auth</code>	Cookie 無しで 401
<code>test_correct_success</code>	フェイク経由で <code>corrected/score/issues</code> が返る
<code>test_correct_empty_text</code>	空文字でも 200、score=100

確認観点 (architecture.md より)

- 設計通り：永続化しない、認証必須、構造化出力
- 環境変数をハードコードしてない (`GOOGLE_API_KEY` / `GEMINI_MODEL` は env から読み込み)
- エラーハンドリング：Gemini エラーは 500 として伝播（隠蔽しない）
- LLM 生成内容を理解している：チェーンの遅延初期化、依存注入、mock 戦略を文書化済み

手動結合テスト（実 Gemini 呼び出し）

```
echo "GOOGLE_API_KEY=実物のキー" >> .env
echo "INITIAL_ADMIN_PASSWORD=admin1234" >> .env
docker compose -f docker/compose.yml up -d
uv run uvicorn api.main:app --reload
# Swagger UI: http://localhost:8000/docs で /api/login → /api/correct を試す
```

次フェーズへの引き継ぎ

phase 4（フロントエンド）で着手するもの：

- SvelteKit の `/login` (shadcn の login-01 を流用)
- SvelteKit の `/` (textarea + 校正ボタン + 結果表示)
- API 呼び出しは `fetch('/api/correct', { credentials: 'include' })` で Cookie を載せる
- 校正中のローディング、エラー時のアラート表示

- ログアウトボタン (layout)

注意点：

- フロントは Cookie ベースなので、SvelteKit 側の `+layout.ts` で `ssr = false` (pure SPA) の方針通り
- バックエンドの CORS 設定は phase 4 で追加 (`allowed_origins` env を新設)
- 401 を受けたら `/login` にリダイレクト (JWT 期限切れ対応)

phase 5 (管理者機能)：

- `POST /api/admin/user`、`PATCH /api/admin/user/{id}/deactivate`
- `Depends(require_admin)` で gate